

算法选择：首先它是一棵树，然后它也没有贪心性质，每个点只有选和不选两种状态，所以树形 dp，这很明显。

直观的想法：设 $f[i,j]$ 表示当前到了以 i 为根的子树，一共放了 j 个人所能达到的最大价值。我们只需枚举 i 选不选以及儿子放多少个人即可。

但是，一个点的儿子好多好多，直接枚举显然是不行的。

改进 1：为了避免一个点超多儿子不好做的情况，我们需要把这个树转成二叉树。左儿子右兄弟法。

改进 2：转成二叉树后，一个点与他本身的父亲疏远了，为了保持父子关系，我们的状态需要改：设 $f[i,j,k]$ 表示当前到了 i 为根的子树，一共放了 j 个人， $k=1$ 表示 i 的原父亲选， $k=0$ 表示 i 的原父亲不选，所能达到的最大价值。

状态转移：对于 i ，我们枚举它的 j ，当 $k=0$ 时， $f[i,j,k]$ 要么 $=f[\text{右儿},j,0]$ （表示从 i 原父亲那里转移到 i 的兄弟），要么 $=\max(f[\text{左儿}, j-\text{self}, \text{self}] + \text{val}[i] * \text{self})$ （表示从 i 原父亲那里转移到 i ），其中 self 表示 i 选不选，选就是 1 不选就是 0。当 $k=1$ 时，枚举左儿子（即原来的儿子）选了 j_1 个， $f[i,j,k] = \max(f[\text{左儿}, j_1, \text{self}] + f[\text{右儿}, j-j_1-\text{self}, 1] + \text{val}[i] * \text{self})$ ，这显然了。

答案： $\max(f[1, m, 0], f[1, m, 1])$

时间：预处理（转成二叉树）需要 n ，枚举 i 需要 n ，枚举 j 需要 m ，枚举儿子选多少需要 m ，然后枚举 k 和 self 可以忽略，总共就是 $O(nm^2)$ 。

——by kqp